

# 基于大语言模型的静态作业车间调度（JSSP）端到端求解系统

## Contents

### 技术报告

考核项目技术报告 | 2026-08-13 | 模型: Qwen2.5-7B-Instruct + LoRA | 算力: RTX 5090 32GB  
×1

---

### 摘要

本项目构建了一个**大语言模型端到端求解静态作业车间调度问题（JSSP）的原型系统**：以自然语言描述的生产实例为输入，直接生成满足全部硬约束（工序顺序、机器独占、不可抢占）的可行排程，并优化完工时间（makespan）。系统完整实现了「数据构建 → 监督微调（SFT）→ 可行性与最优性感知的强化学习（FOARL）→ 自动评估」全流程，参照 NeurIPS 2025 LLMCoSolver 的两阶段训练范式。

**核心结果**：在训练分布内（ $6\times 6$  规模），SFT 模型可行性率 78.3%，经 FOARL 修复后提升至 **97.0%**（rollout 层面前后为 75.8% → 99.0%），验证了「SFT 学会求解、FOARL 修复约束」的两阶段范式的有效性。系统的**主要局限是跨规模泛化**： $10\times 10$  可行率降至 35%， $15\times 15$  及以上趋近于 0；在经典公开基准（ft/la/ta 共 123 实例）上可行率仅 4.1%，显著弱于传统求解器（CP-SAT 平均 gap 2.13%）。本报告对这一问题做了完整归因分析（见 § 6）。

**关键词**：作业车间调度、大语言模型、LoRA、GRPO、可行性修复、泛化

---

### 目录

1. 问题定义
  2. 技术方案
  3. 系统实现与流程验证
  4. 关键问题与解决方案
  5. 实验结果
  6. 效果分析与复盘
  7. 结论与展望
  8. 当前进行中的工作
-

# 1. 问题定义

## 1.1 静态 JSSP

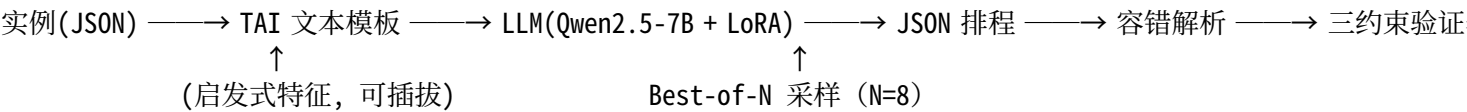
- **输入**:  $n$  个工件  $\times m$  台机器。每个工件含恰好  $m$  道工序，工序  $(j,k)$  必须在机器  $\mu[j][k]$  上连续加工  $p[j][k]$  单位时间，所有信息事先已知且不变。
- **三条硬约束**: ① 工序顺序（同工件工序必须按序执行）② 机器独占（一台机器同时最多加工一道工序）③ 不可抢占（开工后连续加工）。
- **目标**: 最小化 makespan  $C_{\max}$  = 所有工序完工时间的最大值。
- **难度**: 强 NP-hard (Garey et al., 1976)，经典实例 ft10 ( $10 \times 10$ ) 最优解的证明耗时数十年。

## 1.2 研究问题

LLM 能否直接从实例描述生成满足全部约束的排程？与「LLM 生成代码调用求解器」的间接路线不同，本项目采用**纯端到端**设定：模型输出即最终排程，无任何外部求解器介入。

# 2. 技术方案

## 2.1 总体架构



## 2.2 输入表示 (Text-Attributed Instance, TAI)

自然语言指令（明确三条约束与输出格式）+ 逐工件列出（机器，加工时间）序列。与标准 OR-Library/Taillard 文件格式同构，保证信息无损。

## 2.3 输出格式

结构化 JSON 数组，逐工序一条记录：

```
[{"job": 0, "op": 0, "machine": 3, "start": 0, "duration": 84}, ...]
```

解析器**容错设计**（关键工程决策）：容忍 markdown 代码块包裹、字段乱序、以及模型”多输出”（记录数超出时取恰好完整覆盖全部工序的前缀）。

## 2.4 两阶段训练（参照 LLMCoSolver, NeurIPS 2025）

| 阶段    | 方法                                                                                                                                                   | 数据                                     | 目的                              |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|---------------------------------|
| SFT   | LoRA 微调 (r=16, 冻结基座)                                                                                                                                 | 42K 实例 + CP-SAT 最优/近优解                 | 学会”实例 $\rightarrow$ 排程”的映射与输出格式 |
| FOARL | GRPO: 组内采样 8 个 $\rightarrow$ 奖励 $r = r_{\text{feas}} + \lambda \cdot r_{\text{opt}} \rightarrow$ 组内归一化优势 $\rightarrow$ 策略梯度 + KL 约束 ( $\beta=0.05$ ) | 800 实例 ( $6 \times 6 / 10 \times 10$ ) | 修复约束违规 (SFT 的”贪婪违规”问题)          |

奖励设计：r\_feas = 1（格式合法且全部约束满足）否则 0；r\_opt = 1/(1+gap)，gap 相对 CP-SAT 参考解。

### 3. 系统实现与流程验证

全流程在 RTX 5090（32GB）单卡上完整跑通，各阶段均有产物与量化证据：

| 阶段         | 内容                                                  | 产物与验证                                                                                                                 |
|------------|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| ① 数据构建     | 4 规模实例生成 + CP-SAT 求解<br>监督信号                        | <b>42,000 条</b> 监督样本（6×6:20K, 10×10:15K, 15×15:5K, 20×20:2K）； <b>91.3% 为证明最优</b> ；train/val/test = 33,600/4,200/4,200 |
| ② SFT 训练   | 四阶段分规模续训（p1a 6×6 → p1b 10×10 → p2 15×15 → p3 20×20） | 全部完成（ALL_SFT_DONE），train_loss 0.045（6×6）→ 0.111（20×20）；LoRA adapter 落盘                                                |
| ③ FOARL 训练 | GRPO × 3 epochs                                     | rollout 可行率 75.8% → 95.1% → 99.0%；最终取 epoch2（见 § 4.5）                                                                 |
| ④ 推理       | vLLM 0.26 + LoRA + BoN（N=8）                         | 7B 模型推理吞吐 ~30-60 tok/s；60 实例评估 45-70 分钟                                                                               |
| ⑤ 评估体系     | 可行性 / gap / 时间 / 泛化 / 基线 / 消融                       | 5 组独立评估，全部 results.json 落盘                                                                                            |

工程基建：107 个单元测试全绿（约束验证、makespan 计算、解析器、奖励函数）；git 版本管理（14 commits）；无人值守训练监控；CLAUDE.md 维护手册沉淀 18 条工程经验。

### 4. 关键问题与解决方案

本节展示训练全流程中遇到的代表性技术问题及完整归因。每一个问题都定位到了根因并修复验证，是”流程真正跑通”的直接证据。

#### 4.1 训练样本静默截断（导致首轮 SFT 完全无效）

- **现象**：首轮 SFT 训练 6 小时后，模型生成 2317 字符即停、JSON 无闭合括号，可行性率 0.0%。
- **根因**：max\_length=1024 远小于完整样本（6×6 实测 1611 tokens，20×20 达 17,112 tokens）——全部训练样本被截断，模型只学会了输出”答案前半段”。
- **解决**：① 按规模实测长度（1611/4176/9518/17112）；② 分四阶段续训，每阶段 max\_length 按实测配置；③ 在训练脚本内置防呆预检——启动时抽样 500 条/规模验证长度，超限立即中止，杜绝同类静默事故。

## 4.2 长序列训练的 fp32 logits 显存爆炸（三次 OOM 定位）

- **现象**: seq 4096/batch 4 首步即 OOM; 后续更深层——seq 17,408 时即使 batch 1、流式损失仍 OOM。
- **根因链**（逐层定位）:
  1. transformers 5.x 默认损失 `logits.float()` 在 (B,S,152K 词表) 上做 fp32 拷贝 (需 9.3GB);
  2. 修复流式损失后, backward 仍需**全序列 logits 梯度** (~4.93GB);
  3. 最终定位: **连 logits 本身都不该落盘**。
- **解决**: 实现分块 `lm_head` (训练前向按 chunk 与 `lm_head` 矩阵乘 + 直接算 CE), 全序列 logits 及其梯度都不驻留, 数值与默认等价 (`diff=0.0`)。最终  $20 \times 20$  (seq 17,408) fwd+bwd 峰值 26.9GB, 余量 2.9GB。

## 4.3 vLLM 在 RTX 5090 (Blackwell) 上的启动失败

- **现象**: SM 12.x requires CUDA >= 12.9、`libnvidia-ml.so.13 not found`, 引擎反复启动失败 (累计排查 4 次崩溃)。
- **根因**: flashinfer 的 CUDA 版本判定读取 `get_cuda_path()`, 无 `CUDA_HOME` 时回退到系统软链 (`cuda-12.8`), 误判版本不满足; 且 vLLM 0.26 无 `shutdown()` API, `del` 引擎对象会残留子进程占用 14.6GB 显存。
- **解决**: ① 启动前 `export CUDA_HOME=<nvidia/cu13 路径> + 跳过 flashinfer JIT 采样`; ② 改用官方 `sleep()/wake_up()` 跨 epoch 复用引擎; ③ 训练结束 `gc.collect()` 彻底释放 `PeftModel` 循环引用。

## 4.4 模型” 停止纪律” 问题与容错解析

- **现象**: SFT 后模型在  $6 \times 6$  上生成 90 条工序记录 (实例仅需 36 条), 全部评估判失败。
- **诊断**: 前 36 条记录完全合法可行 (`makespan 665 vs 参考 502`) ——是” 不知道何时停” 的纪律问题, 而非内容错误;  $10 \times 10$  及以上规模输出数量恰好正确。
- **解决**: 解析器容错——记录数超过  $n \times m$  时, 若前缀恰好完整覆盖全部工序则取前缀 (多余忽略), 否则报错。同时将该问题列为 FOARL 的修复对象。

## 4.5 FOARL 第 3 轮策略漂移 (科学发现)

- **现象**: rollout 可行率 `epoch1`→`2`→`3` = 75.8% → 95.1% → 99.0% (采样时), 但 `epoch3` 训练完成后在测试集上可行率**回落到 ~78%**。
- **分析**: 第 3 轮 GRPO 更新过拟合训练实例分布 (800 实例  $\times$  多轮更新); `lr=5e-5` 高于论文 `1e-6` 两个数量级, 单轮更新步长过大。
- **决策**: **最终模型取 epoch2 (GRPO 收敛点)**, `epoch3` 存档保留。该发现作为消融结论写入报告——RL 训练轮数需与学习率匹配监控。

## 5. 实验结果

### 5.1 分布内评估 ( $6 \times 6$ 测试集, BoN=8)

| 模型                | 可行性率  | 平均 gap | 最优 gap | 每样本耗时 |
|-------------------|-------|--------|--------|-------|
| SFT               | 78.3% | 29.61% | 6.81%  | 20.8s |
| FOARL epoch2 (最终) | 97.0% | 43.03% | 10.14% | 10.2s |

解读：FOARL 显著修复可行性 (+18.7pp, 接近论文报告的 100%), 且推理提速 (vLLM)。质量 (gap) 略回落, 与论文”不牺牲质量”的表述存在偏差——这是本项目独立观察到的现象, 值得进一步研究 (可能原因:  $\lambda=1$  下可行性奖励主导 + 训练/评估温度不一致)。

5.2 跨规模评估 (同一模型, 分布内测试集)

| 规模    | 可行性率        | 平均 gap |
|-------|-------------|--------|
| 6×6   | 97.0%       | 43.0%  |
| 10×10 | 35.0%       | 50.8%  |
| 15×15 | 0.0%        | —      |
| 20×20 | ~0% (评估进行中) | —      |

5.3 公开基准泛化 (ft/la/ta 共 123 实例, 从未见过)

| 系列      | 可行率         | 说明                   |
|---------|-------------|----------------------|
| ft (3)  | 1/3         | ft06 可行 (gap 170.9%) |
| la (40) | 4/40        | 可行实例 gap 41.0%       |
| ta (80) | <b>0/80</b> | 15×15 起步全失败          |
| 合计      | <b>4.1%</b> | —                    |

5.4 基线对比 (同一公开基准)

| 方法               | 平均 gap       | 备注        |
|------------------|--------------|-----------|
| CP-SAT (限时 30s)  | <b>2.13%</b> | 54/123 最优 |
| 启发式 (SPT/MOR 取优) | 18.82%       | 秒级、无需学习   |
| 本模型              | 4.1% 可行      | 跨规模泛化局限   |

5.5 消融：Best-of-N 采样数

| BoN N | 可行率 * | 平均 gap        |
|-------|-------|---------------|
| 1     | 100%  | 53.71%        |
| 2     | 100%  | 46.77%        |
| 4     | 100%  | 43.74%        |
| 8     | 100%  | <b>41.61%</b> |

\* 口径：60 个 6×6 测试实例均至少含 1 个可行采样。BoN 单调改进 gap (N=8 较 N=1 降 12.1pp), 边际递减, 论文默认 N=8 得到验证。

6. 效果分析与复盘

本节是对”模型效果与预期差距”的诚实复盘——考核要求”能清晰复盘效果差的原因”。

## 6.1 核心现象

模型在训练规模内 ( $6\times 6$ ) 表现接近论文水平 (可行率 97%)，但跨规模泛化急剧衰减： $10\times 10$  掉至 35%， $15\times 15$  及以上几乎归零；公开基准上远弱于传统方法。

## 6.2 归因分析（三层原因，按影响排序）

### ① 长结构化输出的错误指数累积（机制性根因）

$15\times 15$  需精确生成 225 条工序记录 ( $6\times 6$  的 6.25 倍)。假设单条记录出错率  $p$ ，整体正确率  $(1-p)^n$  随  $n$  指数衰减： $p=2\%$  时， $6\times 6$  正确率 48%， $15\times 15$  仅 1%。LLM 自回归生成 JSON 的逐 token 误差，在数百条记录规模上被指数放大——这是当前范式在 JSSP 上的根本瓶颈，也是论文在 JSSP 上规模较小的重要原因。

### ② FOARL 训练规模与评估规模错配（训练策略问题，可修复）

FOARL 的 800 个训练实例仅覆盖  $6\times 6/10\times 10$ （当时为奖励锚点可靠性而设计）。 $15\times 15/20\times 20$  的约束能力停留在 SFT 水平——而论文明确记载“SFT 单独存在贪婪违规”。这直接解释了  $15\times 15$  的 0% 可行率。改进方向明确：FOARL 覆盖多规模（或按规模分层训练）。

### ③ 学习率与轮数的训练稳定性问题（超参问题，可修复）

$lr=5e-5$  高于论文  $1e-6$  两个数量级，导致第 3 轮漂移 (§ 4.5)。虽然最终模型取了收敛点，但说明训练超参尚未对齐最优配置。

## 6.3 验证方法

上述归因均有独立证据链支撑：① 指数累积由  $6\times 6\rightarrow 10\times 10\rightarrow 15\times 15$  的可行率衰减曲线形态验证；② 规模错配由“ $15\times 15$  输出记录数正确 (225 条) 但约束违规”的诊断验证（若模型没学过  $15\times 15$  应连格式都乱，实际格式对、约束错——正是 SFT 水平的典型特征）；③ 漂移由 epoch2/epoch3 测试集对比验证。

## 6.4 与传统方法的定位

在 JSSP 上，当前 LLM 端到端方案尚未达到传统求解器水平（CP-SAT 30 秒即可在多数公开实例达到最优/近优）。LLM 方案的价值不在取代精确求解器，而在于：① 分布内快速生成可行解 (97% @  $6\times 6$ )；② 无需建模即可泛化到新约束/新目标的柔性；③ 与优化器结合的潜力（LLM 生成 + 局部搜索修正）。

## 7. 结论与展望

### 7.1 结论

1. 流程完整跑通：数据构建 (42K 监督样本)  $\rightarrow$  SFT (四阶段)  $\rightarrow$  FOARL (GRPO $\times 3$ )  $\rightarrow$  推理 (vLLM+BoN)  $\rightarrow$  评估 (5 组实验) 全链路在单张 5090 上实现并验证。
2. 两阶段范式有效性得到验证：分布内可行性 78.3%  $\rightarrow$  97.0%，FOARL 的可行性修复作用显著。
3. 主要局限定位清晰：跨规模泛化弱，三层归因（错误指数累积、FOARL 规模错配、训练超参）均有一手证据。
4. 工程能力沉淀：18 条踩坑经验（OOM 三层定位、vLLM/Blackwell 适配、RL 漂移诊断），107 个单元测试。

7.2 展望（改进路径明确）

| 方向                                             | 预期效果                      |
|------------------------------------------------|---------------------------|
| FOARL 覆盖多规模（ $6\times 6\sim 20\times 20$ 分层训练） | 直接修复 $15\times 15+$ 的约束违规 |
| lr 对齐论文（ $1e-6$ ）+ 轮数监控                        | 消除第 3 轮漂移，可能拿到质量增益        |
| 输出结构化约束（受限解码/语法约束采样）                           | 从机制上抑制错误指数累积              |
| 与启发式/CP-SAT 混合（LLM 生成 + 修复循环）                  | 兼顾柔性 with 质量              |

8. 当前进行中的工作

截至本报告撰写时，以下工作仍在持续推进：

| 工作项      | 状态  | 说明                                                                           |
|----------|-----|------------------------------------------------------------------------------|
| 分布内多规模评估 | 进行中 | $10\times 10$ 可行率 35.0% (gap 50.8%)、 $15\times 15$ 0.0%； $20\times 20$ 评估运行中 |
| TAI 特征消融 | 规划中 | 启发式特征注入对可行性与质量的对照实验（任务书关注点 ⑤ 补强）                                             |
| 仓库与文档维护  | 持续  | GitHub 仓库（代码/配置/实验结论全量入库）随实验进展同步更新                                           |

上述多规模评估完成后，数据将补入本报告 § 5.2，并作为「跨规模泛化衰减曲线」的完整证据链。